Phone Number Mnemonics

AlgoExpert · Medium · Recursion

The problem

- All letter strings a digit string could spell on a keypad (0 and 1 map to themselves).

Intuition

- Each digit maps to a few letters; explore every combination with backtracking.

Brute force vs optimal

- Iterative: expand a list digit by digit (BFS)
- Backtracking with a StringBuilder (DFS)
- Both $O(4^n \cdot n)$

Approach

- 1. Recurse digit by digit
- 2. Try each letter for the digit
- 3. At the end, record the word

Worked example

digit	letters
1	1
9	wxyz
0	0
5	jkl

1905 → letters per digit

Worked example

```
1 → 1w → 1w0 → 1w0j, 1w0k, 1w0l

       → 1x0 → 1x0j …
```

Depth-first: one letter per digit

Worked example

```
1w0j, 1w0k, 1w0l, 1x0j, 1x0k, 1x0l

1y0j, 1y0k, 1y0l, 1z0j, 1z0k, 1z0l
```

12 mnemonics = 1 × 4 × 1 × 3

Java solution

```java
public class PhoneNumberMnemonics {
    private static final String[] KEYS = {"0", "1", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};

    public static List<String> phoneNumberMnemonics(String phone) {
        List<String> out = new ArrayList<>();
        build(phone, 0, new StringBuilder(), out);
        return out;
    }

    private static void build(String p, int i, StringBuilder sb, List<String> out) {
        if (i == p.length()) { out.add(sb.toString()); return; }
        for (char c : KEYS[p.charAt(i) - '0'].toCharArray()) { sb.append(c); build(p, i + 1, sb, out); sb.setLength(sb.length() - 1); }
    }
}
```

Complexity

- Time: O(4^n * n)
- Space: O(4^n * n)

Edge cases & pitfalls

- 0 and 1 map to themselves here
- Undo the append after recursing
- Empty input → [""]

Interview tips & follow-ups

- State the output-size bound
- Follow-up: only dictionary words (use a trie)

Recap

- Each digit maps to a few letters; explore every combination with backtracking.
- Complexity: O(4^n * n)